

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description:

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. PocketSmart AI addresses this challenge through a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services.

Using Gemini 1.5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. It supports interior design planning, party planning, and matching jewelry selection while keeping recommendations within defined budgets.

PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through smart planners, adaptive forms, and a recommendation engine that bridges multiple categories and e-commerce ecosystems.

Scenarios

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users select the Home Budget Planner, enter a budget, specify room types and quantities for items such as lights, ceiling fans, and dining tables, and receive cost-effective options from platforms such as IKEA and Amazon. Recommendations balance functionality, style, and price.

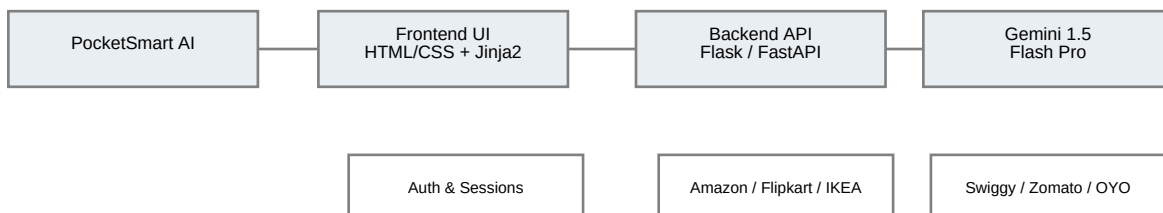
Scenario 2: AI-Based Party Budget Planning

Users enter total budget, guest count, event type, and venue details. The AI allocates the budget across catering, decoration, and entertainment, with sources including Swiggy, Zomato, and OYO.

Scenario 3: Jewelry Recommendations for Occasions

Users enter budget, occasion, and style preferences and can optionally upload an outfit image. Gemini analyzes the inputs and offers jewelry options from platforms such as Amazon and Flipkart, aligned with the occasion, style, and budget.

Technical Architecture:



Description: PocketSmart AI is a modular, budget-aware recommendation platform. The supplied project documentation describes a Flask backend and later implementation milestones describe FastAPI; both are retained here. Gemini 1.5 Flash Pro processes text, numbers, and optional images to generate structured recommendations.

Core technologies: Flask / FastAPI, Gemini 1.5 Flash Pro, third-party platform data sources, HTML/CSS, JavaScript, and Jinja2.

Pre-requisites:

- IBM Cloud Account Setup: IBM Cloud Console.
- Watsonx Access and Configuration: Watsonx Overview.
- Python Basics: Python.org.
- FastAPI Framework: FastAPI Docs.

The project also requires Gemini API access and a configured Python development environment.

Project Workflow:

Milestone 1: Gemini AI Initialization

- **Activity 1.1:** Set up Google Cloud access and Gemini 1.5 Flash Pro API access.
- **Activity 1.2:** Configure API keys, usage limits, and multimodal testing.
- **Activity 1.3:** Fine-tune prompts for budget interpretation and product recommendations.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Design the backend structure for routing, sessions, and planner modules.
- **Activity 2.2:** Create Home, Party, and Jewelry endpoints.
- **Activity 2.3:** Integrate Gemini prompts and templates with optional image inputs.

Milestone 3: Backend – FastAPI Integration

- **Activity 3.1:** Define planner, authentication, token, session, recommendation-detail, history, startup, and main routes.
- **Activity 3.2:** Organize services for recommendation logic, AI calls, and input/output handling.

- **Activity 3.3:** Configure CORS, static routing, authentication, and session handling.

Milestone 4: UI Development

- **Activity 4.1:** Build responsive forms for Home Interior, Party, and Jewelry planners.
- **Activity 4.2:** Add dynamic AI result display using JavaScript and Jinja2.

Milestone 5: Testing & Optimization

- **Activity 5.1:** Test real-world scenarios across all three planners.
- **Activity 5.2:** Evaluate response quality, budget adherence, and platform accuracy.
- **Activity 5.3:** Optimize prompts and validate edge cases.
- **Activity 5.4:** Add fallback recommendations when results are insufficient.

Milestone 1: Gemini AI Initialization

This stage establishes the AI infrastructure required to power PocketSmart AI. Gemini 1.5 Flash Pro interprets user inputs, processes budgets, generates recommendations, and performs image analysis for the Jewelry Planner.

Activity 1.1: Set up Google Cloud account and enable Gemini API

- Sign in or create a Google Cloud account and obtain an API key.
- Select the required model through the available model interface or API documentation.

Activity 1.2: Configure access and permissions

- Create the API key and copy it securely for application configuration.
- Protect the key and use environment configuration rather than hard-coding it.

Activity 1.3: Validate Gemini API connectivity

- Make a sample Python prompt call to confirm integration.
- Test text-only and image + text prompts for multimodal compatibility.

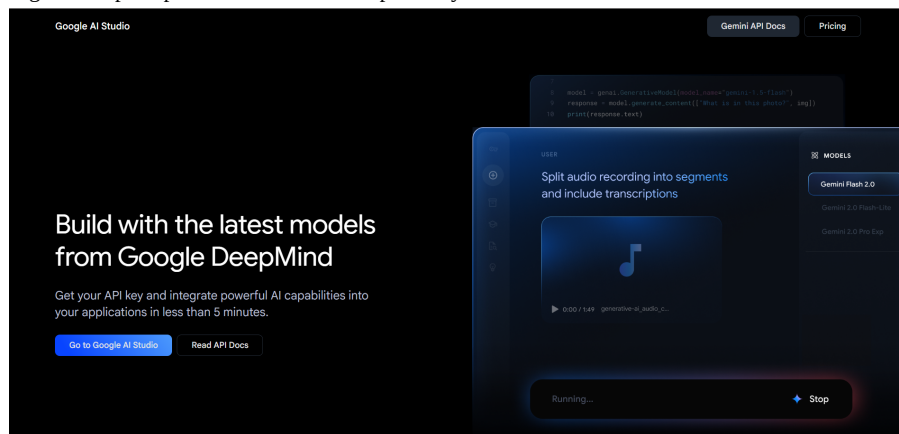


Figure: Example AI integration/development view included in the supplied PocketSmart project material.

Milestone 2: Core Functionalities Development

The backend accepts user budgets and optional images, interprets context, and returns domain-specific, platform-aware recommendations. Core logic covers prompt generation, AI communication, planner-specific logic, and fallback strategies.

Activity 2.1: FastAPI Initialization and import libraries

- Import required libraries, initialize the API application, and load environment configuration.

Activity 2.2: Main Functionality Code Routings

- **/generate-home:** Generates home interior recommendations for furniture, decor, and lighting.
- Use recommendation functions and platform sources such as Amazon, Flipkart, and IKEA.
- **/generate-party:** Suggests catering, venue, and decoration options.
- **/generate-jewelry:** Accepts text and image inputs to return style-matched jewelry.



Figure: Architecture / implementation visual from the PocketSmart project source.

Activity 2.3: Login and Register Page Routings

- **/login:** Authenticates credentials and initiates a session.
- **/register:** Handles new user registration.
- **/logout:** Terminates the current session.

Activity 2.4: Modular Code Structure

- **/token:** Issues a secure JWT token after authentication.
- **/session-info:** Retrieves current session metadata.
- **/session-data:** Returns session-specific data for personalization and recommendation tracking.

Milestone 3: Backend – FastAPI Integration (main.py)

The FastAPI backend bridges the frontend with the Gemini recommendation engine. It provides modular endpoints for Home, Party, and Jewelry planning, plus CORS, sessions, and structured input/output models.

Activity 3.1: Define FastAPI Routes

- **Home Planner:** Accepts home preferences and budget.
- **Party Planner:** Processes party details and guest count.
- **Jewelry Planner:** Uses text and optional image inputs.
- **User Auth:** /register, /login, /logout, /token.
- **Session Info:** /session-info and /session-data.

Activity 3.2: Modular Architecture Setup

- **/recommendations-details:** Returns detailed AI-generated recommendations.

Activity 3.3: CORS & Static Routing

- Configure cross-origin access and static application resources.

Activity 3.4: Startup and Main Function

- **/startup:** Initializes application services and configuration.
- **__main__:** Starts the server with uvicorn when run directly.

Recommendation History

- **/history:** Retrieves past recommendation queries and results for review or reuse.

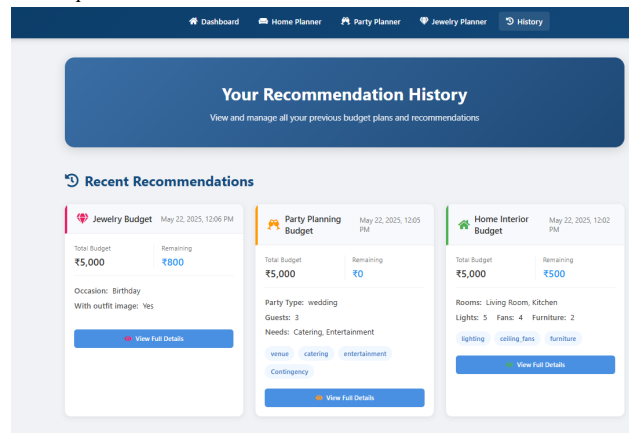


Figure: Recommendation History page from the supplied project material.

Milestone 4: UI Development

The frontend uses HTML, CSS, and Jinja2 templates to provide an intuitive interface for budget-based recommendations across Home Interior, Party Planning, and Jewelry categories.

Activity 4.1: Develop HTML Frontend Interface

- Design the home page with budget inputs and category selectors.
- Implement planner-specific forms connected to backend POST routes.
- Display AI responses in structured card-like layouts with product details and prices.

Website Pages

- **Home Page:** Introduces PocketSmart AI and planner modules.
- **Testimonials Page:** Shows user reviews and success stories.
- **Register / Login:** Supports account creation and authentication.
- **User Dashboard:** Displays recent recommendations, saved queries, and personalized insights.
- **Recommendation History:** Displays previous recommendation queries and results.

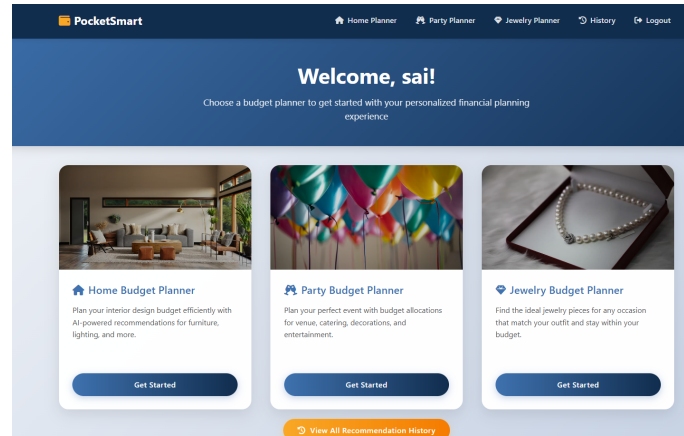


Figure: PocketSmart AI interface visual from the supplied project material.

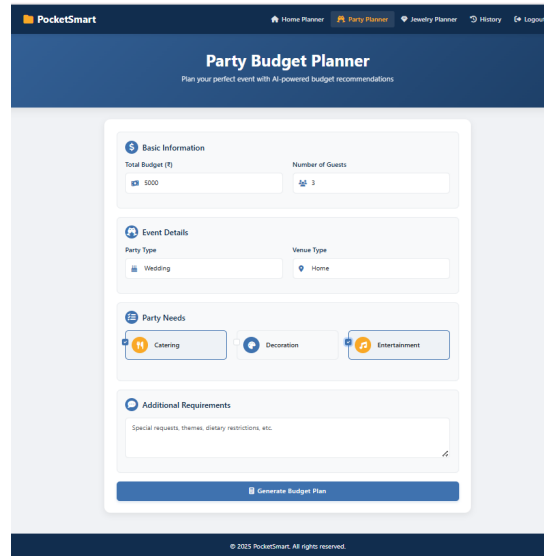


Figure: User Dashboard view from the supplied project material.

Planner Interfaces and Recommendations

Home Interior Budget Planner

Lets users input home decor preferences and budget to obtain tailored suggestions from multiple platforms.

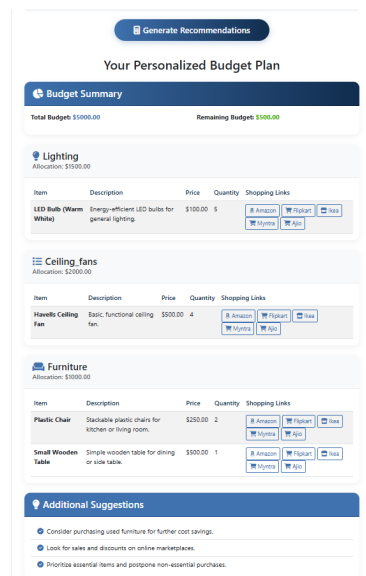


Figure: Home Interior Budget Planner page.

Jewelry Budget Planner

Provides recommendations aligned with occasion and outfit style, with optional image input.

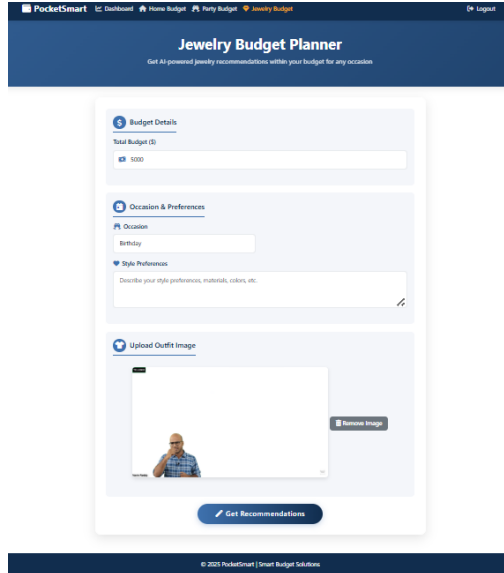


Figure: Jewelry Budget Planner page.

Home Interior Recommendations

Displays AI-generated home-decor product suggestions based on budget and style preferences.

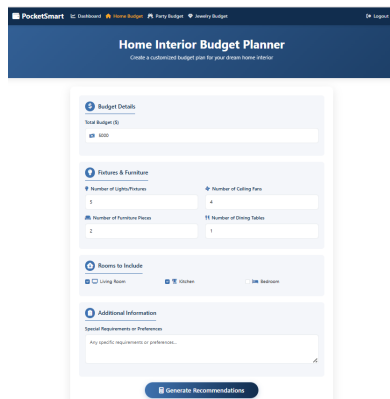


Figure: Home Interior Recommendations page.

Milestone 5: Testing & Optimization

Testing validates reliability, accuracy, and contextual awareness across Home Interiors, Party Planning, and Jewelry Suggestions. Optimization includes prompt refinement, session handling, and UI/UX improvements.

Activity 5.1: Test with Real-World Budget Inputs

- Home decor setups from IKEA and Amazon.
- Party packages including food from Swiggy/Zomato, venues such as OYO, and decoration.
- Jewelry suggestions matched with outfit styles, occasions, and image inputs.

Activity 5.2: Evaluate AI Response Quality

- Check budget adherence, contextual relevance, and platform accuracy.

Activity 5.3: Optimize Prompts and Validation

- Refine prompts and add validation to reduce edge-case failures.

Activity 5.4: Add Fallback Recommendations

- Provide fallback/default recommendations when AI returns insufficient results.

Conclusion:

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across home interiors, party planning, and jewelry selection.

By combining FastAPI with Gemini 1.5 Flash Pro, the system processes budgets, preferences, and images to generate context-aware suggestions from platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more.

From backend architecture to AI integration and a responsive frontend, the project focuses on modular routing, authentication, session handling, and user-centric interaction. Each module is designed to simplify decision-making through automation and personalization.

With its combination of generative AI and web technologies, PocketSmart AI demonstrates how AI can enhance everyday decision-making by bringing fast, personalized, and budget-conscious recommendations to users.

This documentation was reorganized from the supplied PocketSmart AI project document and formatted to follow the structure and presentation style of the supplied sample documentation.